

M O D U L E 0 1

Why TypeScript & The Type System

Designed for engineers who know JavaScript deeply and have zero experience with static typing.



~45 min read



Beginner -> TS



No prior TS needed

1. What Actually Is TypeScript?

TypeScript is JavaScript with a type system layered on top. That's it. Every single JavaScript file you've ever written is valid TypeScript. TypeScript doesn't replace JavaScript — it compiles down to it.

The Golden Rule

TypeScript only exists at development time. When your code runs in Node or the browser, it's plain JavaScript. TS catches your mistakes before runtime, not during.

Think of TypeScript like a spell-checker for your code. Your spell-checker doesn't change what you write — it just tells you when something looks wrong. TS does the same thing for types.

2. The Problem TypeScript Solves

Let's look at a real bug that TypeScript prevents. You've seen something like this in JS:

```
✗ JavaScript — bug discovered at runtime
// JavaScript — this looks innocent

function getUserAge(user) {
  return user.age * 2;
}

const result = getUserAge({ name: 'Alice' });
console.log(result); // → NaN 🤖
```

You only find that bug when the code runs. Maybe it's in production. Maybe it's 2am. TypeScript catches it instantly:

```
✓ TypeScript — error caught at write time
// TypeScript — error shows before you even run the code

interface User {
  name: string;
  age: number;
}

function getUserAge(user: User) {
  return user.age * 2; // ✓ TS knows age is a number
}

getUserAge({ name: 'Alice' });
// TS Error: Property 'age' is missing in type '{ name: string }'
// You see this error in your editor — before running anything ✓
```

3. Primitive Types — The Building Blocks

TypeScript has the same primitive types as JavaScript. The difference is you declare them explicitly.

JavaScript	TypeScript
<code>let name = 'Alice';</code>	<code>let name: string = 'Alice';</code>
<code>let age = 30;</code>	<code>let age: number = 30;</code>
<code>let isActive = true;</code>	<code>let isActive: boolean = true;</code>
<code>let data = null;</code>	<code>let data: null = null;</code>
<code>let value = undefined;</code>	<code>let value: undefined = undefined;</code>
<code>const id = Symbol('id');</code>	<code>const id: symbol = Symbol('id');</code>
<code>const big = 9007n;</code>	<code>const big: bigint = 9007n;</code>

💡 Type Inference — TS is smarter than you think

In most cases you DON'T need to write the type. TypeScript infers it automatically. Write: `let name = 'Alice'` and TS already knows it's a string. You only need explicit types when TS can't figure it out — like

function parameters.

4. Your First Type Annotations

Here's the syntax pattern. The colon (:) is how you tell TypeScript what type something is:

Type Annotations in action

```
// Pattern:  variableName: Type = value

let username: string = 'alice_dev';
let score:    number = 9001;
let verified: boolean = false;

// Once typed, TS enforces it everywhere
username = 42;    // ❌ Error: Type 'number' is not assignable to 'string'
username = 'bob'; // ✅ Fine

// Arrays: put [] after the type
let scores: number[] = [10, 20, 30];
let tags:  string[] = ['ts', 'js', 'react'];

scores.push('hello'); // ❌ Error: Argument of type 'string' not
                      // assignable to 'number'
scores.push(99);      // ✅ Fine
```

5. The 'any' Type — Your Escape Hatch (Use Sparingly)

TypeScript has a type called any. It turns off type checking completely for that variable — essentially making it behave like JavaScript:

any — the escape hatch

```
let data: any = 'hello';
data = 42;      // ✅ allowed
data = true;    // ✅ allowed
data.foo.bar.baz; // ✅ no error... but will crash at runtime 💣
```

⚠️ **Warning: any defeats the purpose of TypeScript**

Using any is like turning off your spell-checker for a word because it's hard to spell. It works, but you lose all the protection. Use it only when migrating old JS code or working with truly unknown data. Never use it by default.

6. Union Types — 'This OR That'

One of the most powerful features in TypeScript. A union type lets a variable hold one of several types, using the | (pipe) symbol:

Union Types

```
// This variable can be a string OR a number
let id: string | number;

id = 'abc-123'; // ✓
id = 42;        // ✓
id = true;      // ✗ boolean not in the union

// Real-world example: API response can be data OR null
let user: User | null = null;

// Union with string literals (very useful!)
let direction: 'north' | 'south' | 'east' | 'west';
direction = 'north'; // ✓
direction = 'up';    // ✗ 'up' is not in the union

let status: 'loading' | 'success' | 'error';
// TS will autocomplete these values in your editor ✨
```

7. Type Aliases — Naming Your Types

When you find yourself writing the same type over and over, give it a name using the type keyword:

Type Aliases

```
// Without alias – repetitive
let status1: 'loading' | 'success' | 'error';
let status2: 'loading' | 'success' | 'error';
let status3: 'loading' | 'success' | 'error';

// With alias – clean and reusable ✓
```

```
type Status = 'loading' | 'success' | 'error';

let status1: Status;
let status2: Status;
let status3: Status;

// Aliases work for any type – even complex ones
type ID = string | number;
type Coordinates = { x: number; y: number };

const point: Coordinates = { x: 10, y: 20 }; // ✓
```

8. Setting Up TypeScript

Before we write more code, here's how you set up TS in a project:

Terminal — Setup Commands

```
# Install TypeScript globally
npm install -g typescript

# Check version
tsc --version

# In your project folder:
npm install --save-dev typescript
npx tsc --init # Creates tsconfig.json

# Your files: rename .js → .ts
# Then compile:
npx tsc # Compiles all .ts files to .js
npx tsc --watch # Watch mode – recompiles on save
```



Recommended: Use ts-node for quick runs

Install ts-node globally (npm install -g ts-node) to run TypeScript files directly without compiling: ts-node myfile.ts. Great for learning and scripts.

Module Summary

Here's everything from this module at a glance:

- TypeScript compiles to JavaScript — the browser/Node never sees TS
- It catches type errors at write-time, not runtime
- Primitive types: `string`, `number`, `boolean`, `null`, `undefined`
- Syntax: `let name: string = 'Alice'`
- TypeScript infers types automatically in most cases
- Union types: `string | number` — value can be either
- Type aliases: `type Status = 'loading' | 'success' | 'error'`
- Avoid `any` — it removes all type safety

 Next: Module 2

Typing Functions, Objects & Interfaces

Function parameter types, return types, optional params, interfaces vs type aliases, and typing real-world objects.